Advanced Programming Techniques

fbeit
FACHBEREICH ELEKTROTECHNIK
UND INFORMATIONSTECHNIK

Prof. Dr. Michael Lipp

h_da
HOCHSCHULE DARMSTADT
UNIVERSITY OF APPLIED SCIENCES

poll.mnl.de

- **I have understood the basic ideas of OOP**
  1) Fully
  2) Mostly
  3) Good
  4) Barely
  5) Not at all

poll.mnl.de

- **I remember the syntax for defining a class and implementing methods**
  1) Fully
  2) Mostly
  3) Good
  4) Barely
  5) Not at all

Constructor and Destructor

fbeit
FACHBEREICH ELEKTROTECHNIK
UND INFORMATIONSTECHNIK

Special method(s): Constructor

- **New objects' state is undefined, i. e. instance variables have arbitrary values**
- **Initialization can be ensured by defining constructors**
  - Constructors are member functions that have the same name as the class and no return type
  - Constructors are invoked automatically as side-effect of object creation (that's why returning a value wouldn't make sense)
  - Constructors can have parameters
  - A constructor without parameters is called "default constructor"
- **The constructors' responsibility is to ensure defined state**

Achieving defined state

- Assign values to instance variables in body:

```
SimpleVector::SimpleVector(int sz) {
    size = sz;
    elements = new double[size];
}
```

- Initialize instance variables:

```
SimpleVector::SimpleVector(int sz):
    size{sz}, elements{new double[sz]} {
}
```

- Use default member initializer in class definition:

```
class SimpleVector {
private:
    int size = 10;
    // …
}
```

Constructor's counterpart: Destructor

- **Sometimes actions have to be executed when an object is no longer used (goes out of scope or back to free memory pool)**
  - E.g. the memory allocated for data by a `SimpleVector` instance must be returned to the pool
- **A method that is automatically invoked in these cases is called a destructor**
  - The destructor is a special member function that has the same name as the class with a prefixed tilde (~), no return type and no parameters

Object creation/deletion

On stack

```
if (someCondition) {
    // Variable declaration
    // invokes constructor
    SimpleVector testVector{10};

    testVector.print();

    // Block scoped variable
    // goes out of scope,
    // destructor is invoked
}
```

Track in debugger

On heap

```
// Object allocation
// invokes constructor
SimpleVector* vectOnHeap
    = new SimpleVector{10};

vectOnHeap->print();

// Deallocation invokes
// destructor
delete vectOnHeap;
```

`new` and `delete` replace
`malloc` and `free` from C (more later)

Class members

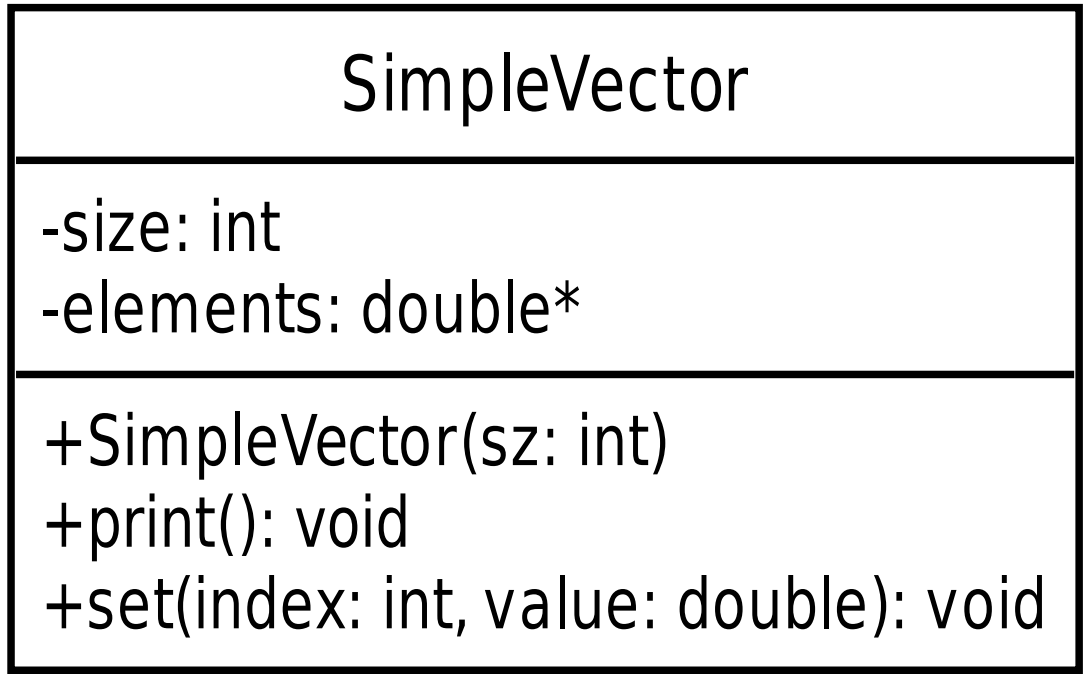
- **Not only objects can have data members and member functions**
- **Classes can have those, too**
  - Defined by prefixing the declaration with "`static`"
  - Class data members are "shared" by all objects, i.e. changes are visible to all instances
  - "public" data members and member functions are accessible from "outside" by prefixing the name with the class name and the scope operator (without having to use an object) (e.g. `cout << MyClass::sampleClassAttribute << endl;`)

UML class diagram

fbeit
FACHBEREICH ELEKTROTECHNIK
UND INFORMATIONSTECHNIK

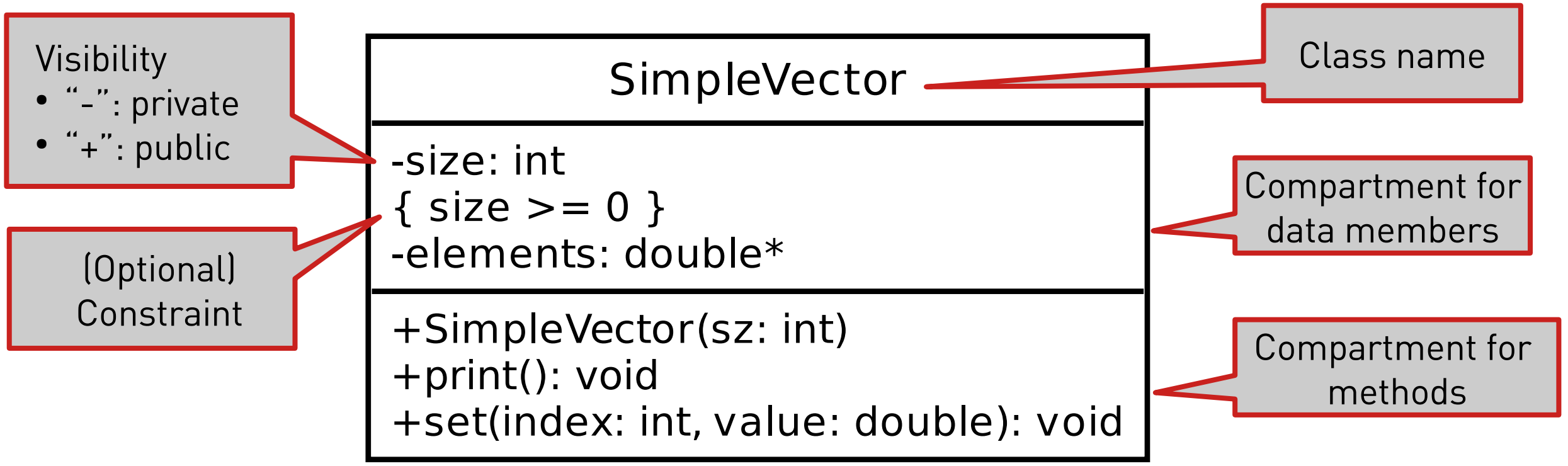
UML class diagram

- "Graphical" representation (for quick overview)



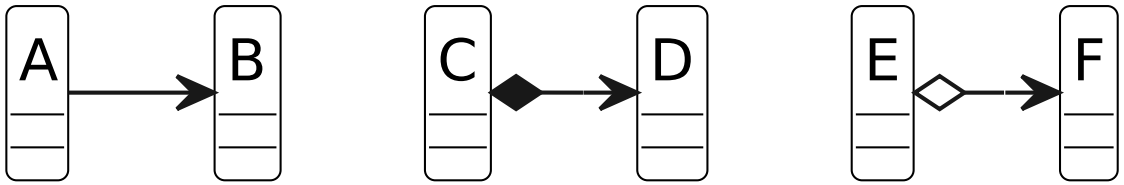

```
class SimpleVector {
private:
    int size;
    double* elements;

public:
    SimpleVector(int sz);
    void print();
    void set(int index, double value);
};
```

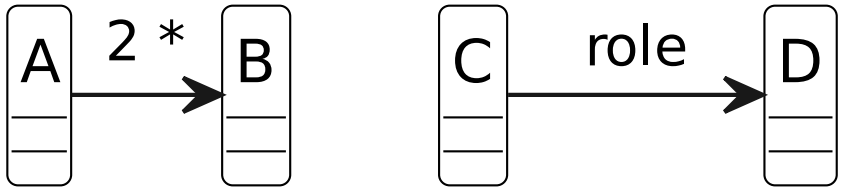
UML class diagram



General rule: First name, then type (separated by colon)
(note usage in variable definition, parameter list, return type)

UML diagram

- Class members are underlined in the diagram
- UML diagrams also depict relationships
  - 

- May be augmented with additional information
  - 

Details: see https://en.wikipedia.org/wiki/Class_diagram

Parameters and return values (Memory I)

fbeit
FACHBEREICH ELEKTROTECHNIK
UND INFORMATIONSTECHNIK

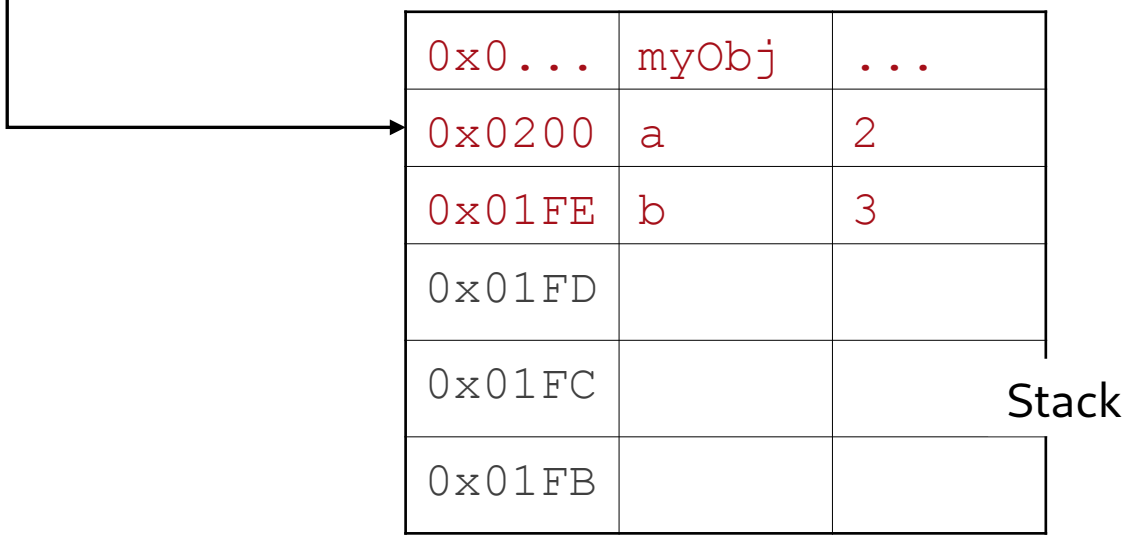
Stack and Heap

- **Like C, C++ distinguishes stack and heap**
  - Stack → automatically managed memory (next slides)
- **Heap → explicitly managed memory**
  - `void* malloc(size_t size)` → `T* new (T)` (or `T* new T[elements]`)
  - `void free(void* allocated)` → `delete allocated` (or `delete[] allocated`)
  - `new` and `delete` call constructor and destructor
    - "`new T[size]`" requires default constructor
  - Calling `new` and `delete` properly is programmer's responsibility
    - Hint: couple it with constructor and destructor (more on this later)

Local variable allocation

```
A myObj;
short a=2, b=3;

myObj.do(a, &b);
```

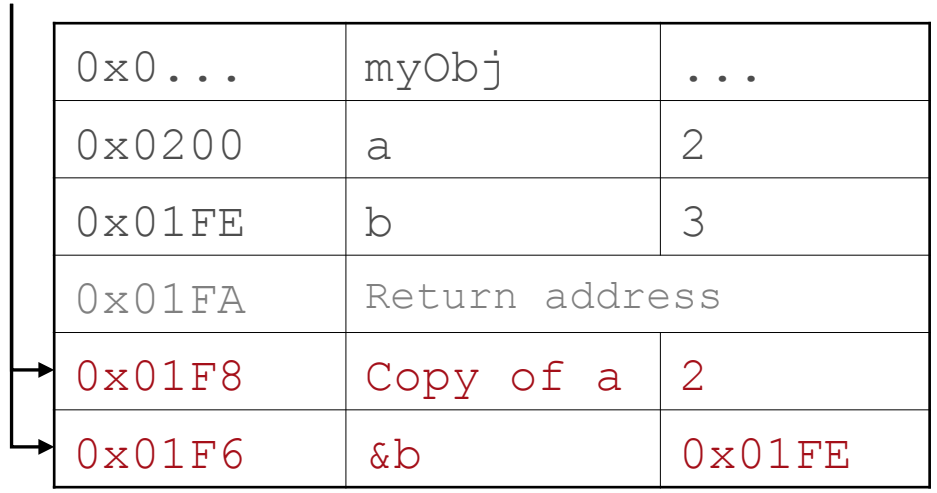
```
void A::do(short x, short* y) {
        short z= x;
        x= *y;
        (*y)++;
}
```



Local variables are stored on the stack, which grows from top to bottom if you think of higher memory addresses being “above” lower addresses

Calling a function

```
A myObj;
short a=2, b=3;
myObj.do(a, &b);
```



0x0...	myObj	...
0x0200	a	2
0x01FE	b	3
0x01FA	Return address	
0x01F8	Copy of a	2
0x01F6	&b	0x01FE

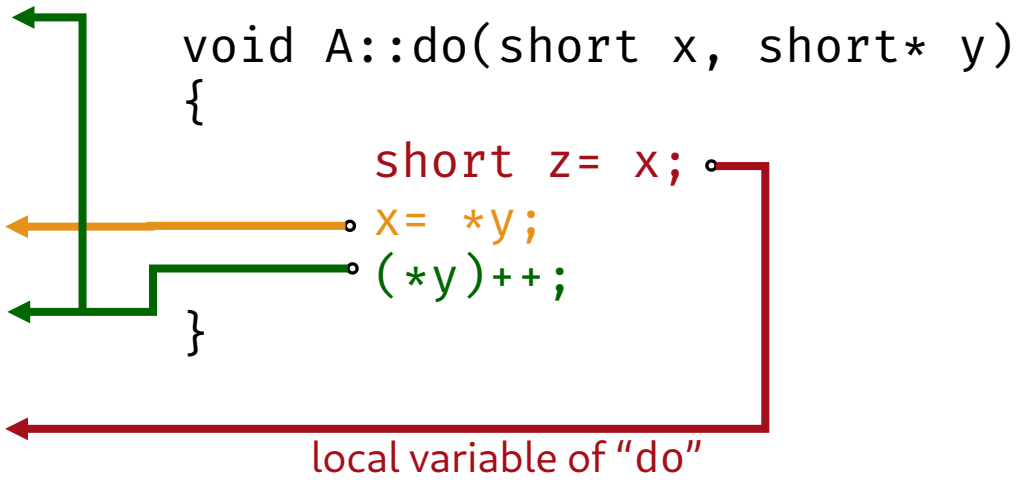
```
void A::do(short x, short* y) {
      int z= x;
      x= *y;
      (*y)++;
}
```

Just like local variables, parameters are stored on the stack when calling a function.

Using parameters while executing

A m
sho
myO

0x0...		
0x0200	a	2
0x01FE	b	3 4
0x01FA	Return address	
0x01F8	Copy of a (a.k.a x)	2 3
0x01F6	&b (a.k.a y)	0x01FE
0x01F4	z	2



Functions "see" only the parameters (and their own local variables)
Passing a pointer, however, allows them to alter a value in the caller's stack frame

After returning from execution

```
A myObj;
short a=2, b=3;
myObj.do(a, &b);
```

```
void A::do(short x, short* y)
t z= x;
y;
;
```

0x0...	myObj	...
0x0200	a	2
0x01FE	b	4
0x01FA		
0x01F8		
0x01F6		
0x01F4		

- Storage space used for parameter values is unused again (may be reused for another call
- Only changes made in caller's storage have an effect
- Return values are returned "temporary objects" (maybe on stack – depends)

Pointer “hiding” in C++: References

- **C++ introduces “pass by reference”**
  - Creation of pointer and dereferencing is “hidden”
    - Declaration: `void A::do(short x, short& y);`
    - Invocation: `myObj.do(a, b);`
  - Exactly the same happens on the stack!
- **References can be considered (and used as) “aliases”**
  - `int i; int& ref=i;` → `i` and `ref` identify the same memory location
- **Reference can also be used for return value**
  - Make sure to only return references to values that stills exist after the function call! (e.g. “z” from the previous example cannot be used for “return by reference”, it’s no longer there after the function call)

Hack: `const` pass by reference

- **Pass by reference avoids copying, however value may be modified by invoked function**
- **Copying may be "expensive" (takes a lot of CPU cycles)**
- **Compromise:**
  - Pass by reference but declare as `const` (unmodifiable)
  - e.g.: `void verbosePrint(const SimpleVector& v);`
- **Drawback:**
  - "`const`" can be cast away

Using `const` has “side effects”

- Try to invoke a method on **`const Vector`**

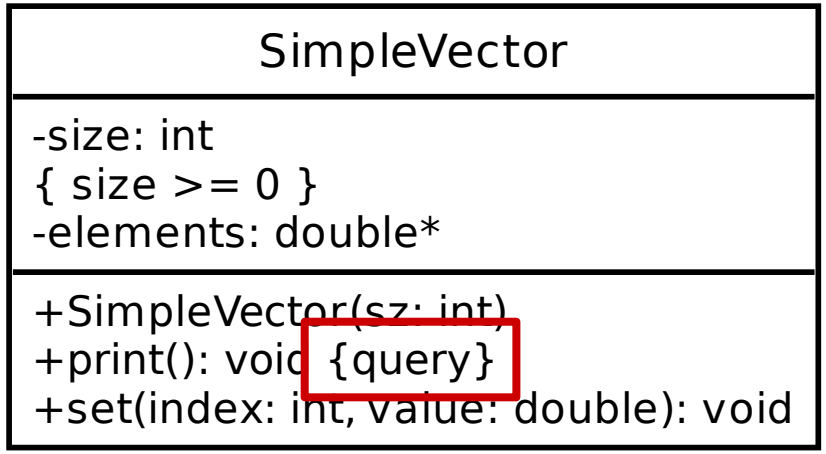
```
void verbosePrint(const SimpleVector& v) {
    cout << "Vector's state:" << endl;
    v.print(); // ERROR
}
```

  - “`error: passing 'const SimpleVector' as 'this' argument discards qualifiers`”
  - Not very intuitive but explainable: `print` is defined for non-const `SimpleVector` only. Invoking `print` on `const SimpleVector` is an attempt to discard the `const` qualifier.

Solution: `const` methods

- **Methods can be defined to work with constant objects (i.e. they don't discard the qualified, they accept it)**
- **Append `const` to method declaration:**

```
class SimpleVector {
    // ...
public:
    // ...
    void print() const;
    // …
};
```



  - Can be interpreted as "this method does not change the object that it is invoked on"
  - Do this from the beginning, not as an afterthought!

Another C++ feature: default arguments

- **C++ introduces default arguments for function parameters**
  - Specified by appending '=' and value to parameter, e.g.

    ```
    SimpleVector(sz=10);
    ```
  - May be provided for trailing parameters only

    ~~`void sample(int p1=5, int p2);`~~ `//` ERROR
  - On invocation, arguments for which default values exist may be "left out", but only from right to left
  - The compiler automatically "inserts" any missing argument using the default value
  - There's no way for the invoked function to know if it receives a default value or an explicitly provided value

Default argument caveats

- **Be aware of possibilities introduced by default arguments**
- **Example Coordinate**

```
class Coordinate {
private:
    int x;
    int y;

public:
    Coordinate(int x=0, int y=0);
};
```

  - Allows "`Coordinate coord1;`" and "`Coordinate coord2{2,3}`" but also "`Coordinate coord3{42};`" which is ambiguous for the reader: x==42, y==42 or both ==42?
  - → Better to define separate default constructor and additional constructor with two parameters